

# CMD Vs. PowerShell

Up to this point, we have discussed the built-in Windows Command-line interpreter `cmd.exe`. Moving forward, we will look at Windows's modern successor to CMD, [PowerShell](#). This section will cover what PowerShell is, the differences between PowerShell and CMD, how to get help within the CLI, and basic navigation within the CLI.

## Differences

PowerShell and CMD are included natively on any Windows host, so we may ask ourselves, "Why would I use one over the other?" Let's address this quickly. Below is a table with some differences between PowerShell and CMD.

### PowerShell and CMD Compared

| Feature             | CMD                                                                 | PowerShell                                                       |
|---------------------|---------------------------------------------------------------------|------------------------------------------------------------------|
| Language            | Batch and basic CMD commands only.                                  | PowerShell can interpret Batch, CMD, PS cmdlets, and aliases.    |
| Command utilization | The output from one command cannot be passed into another directly. | The output from one command can be passed into another directly. |
| Command Output      | Text only                                                           | PowerShell outputs in object formatting.                         |
| Parallel Execution  | CMD must finish one command before running another.                 | PowerShell can multi-thread commands to run in parallel.         |

Most notably, PowerShell has been built to be [extensible](#) and to integrate with many other tools and functionality as needed. Most think of it as just another CLI, but it is much more. Did you know it is also a [scripting language](#)? While CMD has been the default command-line interpreter for Windows hosts only, PowerShell has been released as an [open-source project](#) and has an extensive offering of capabilities that support its use with Linux-based systems as well. Using the [.NET](#) framework has also made PowerShell capable of utilizing an object base model of interaction and output instead of text-based only.

## Why Choose PowerShell Over cmd.exe?

[Why does PowerShell matter for IT admins, Offensive & Defensive Infosec pros?](#)

[PowerShell](#) has become increasingly prominent among IT and Infosec professionals. It has widespread utility for System Administrators, Penetration Testers, SOC Analysts, and many other technical disciplines where ever Windows systems are administered. Consider IT admins and Windows system administrators administering IT environments made up of Windows servers, desktops (Windows 10 & 11), Azure, and Microsoft 365 cloud-based applications. Many of them are using PowerShell to automate tasks they must accomplish daily. Among some of these tasks are:

- Provisioning servers and installing server roles
- Creating Active Directory user accounts for new employees
- Managing Active Directory group permissions
- Disabling and deleting Active Directory user accounts
- Managing file share permissions
- Interacting with [Azure](#) AD and Azure VMs
- Creating, deleting, and monitoring directories & files
- Gathering information about workstations and servers
- Setting up Microsoft Exchange email inboxes for users (in the cloud &/or on-premises)

- Setting up Microsoft Exchange email inboxes for users (in the cloud &/or on premises)

There are countless ways to use PowerShell from an IT admin context, and being mindful of that context can be helpful for us as **penetration testers** and **even as defenders**. As a sysadmin, PowerShell can provide us with much more capability than **CMD**. It is **expandable**, built for **automation** and scripting, has a much more robust security implementation, and can handle many different tasks that CMD simply cannot. As a pentester, many well-known capabilities are built into PowerShell. PowerShell's module import capability makes it easy to bring our tools into the environment and ensure they will work. However, from a stealth perspective, PowerShell's **logging** and **history** capability is powerful and will log more of our interactions with the host. So if we do not need PowerShell's capabilities and wish to be more stealthy, we should utilize CMD.

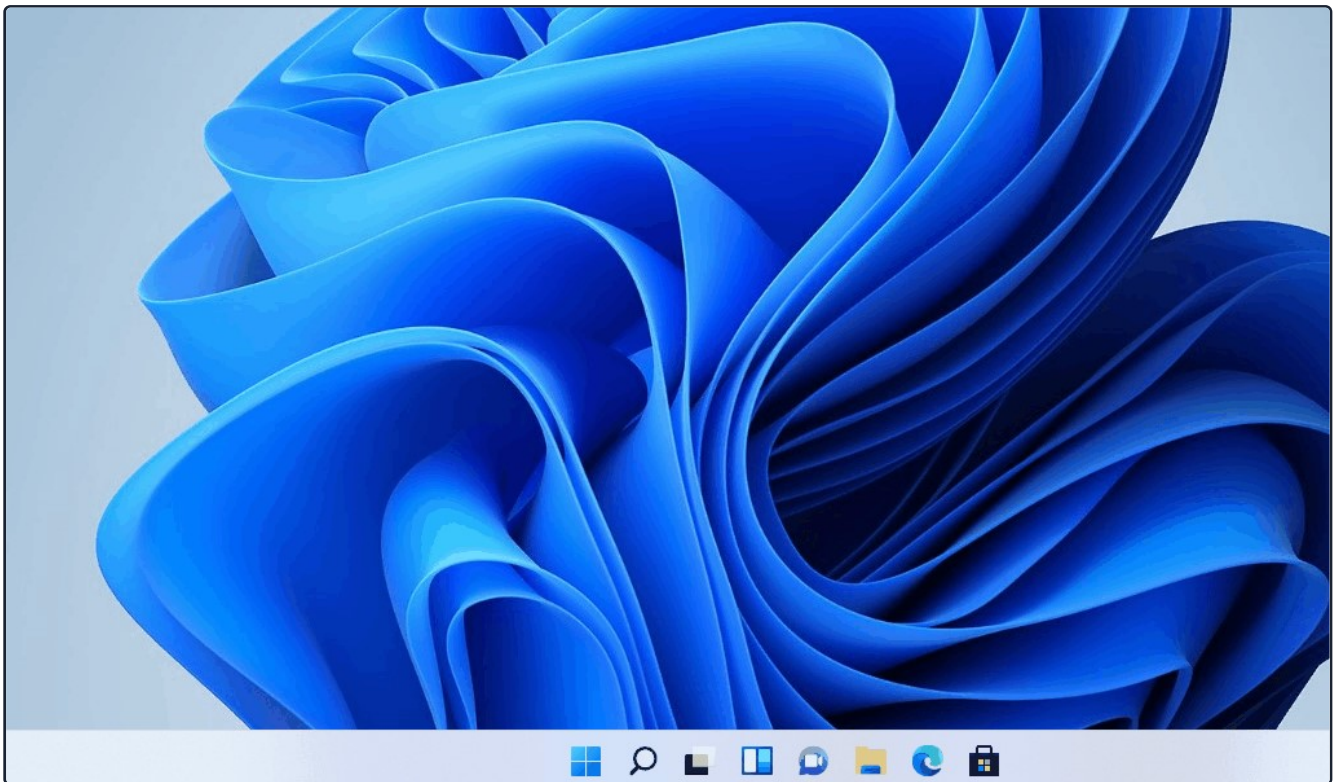
---

## Calling PowerShell

We can access PowerShell directly on a host through the peripherals attached to the local machine or through RDP over the network through various methods.

### 1. Using Windows **Search**

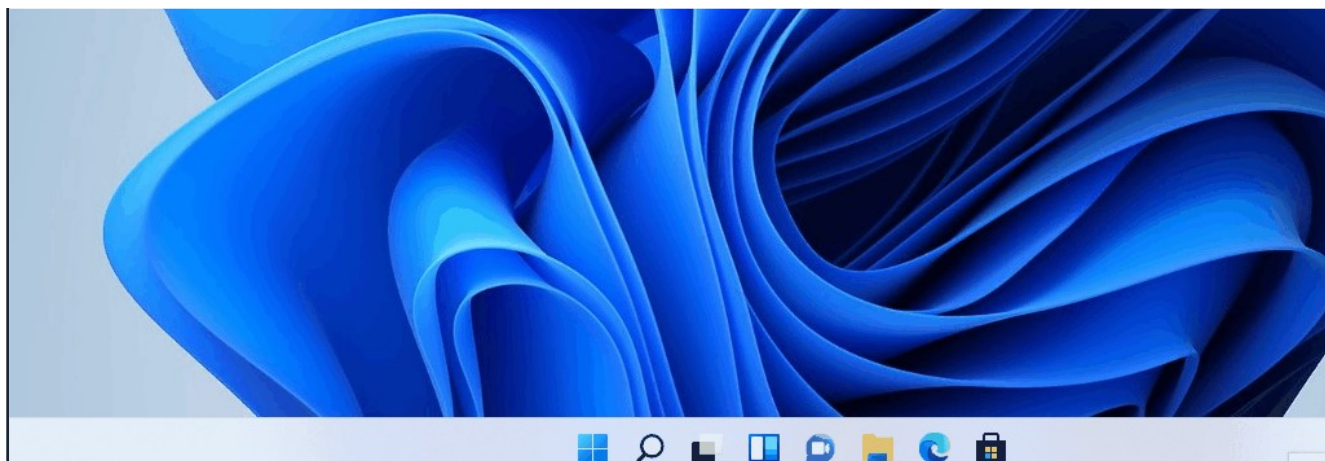
We can type **PowerShell** in Windows Search to find and launch the PowerShell application and command console.



### 2. Using the Windows **Terminal** Application

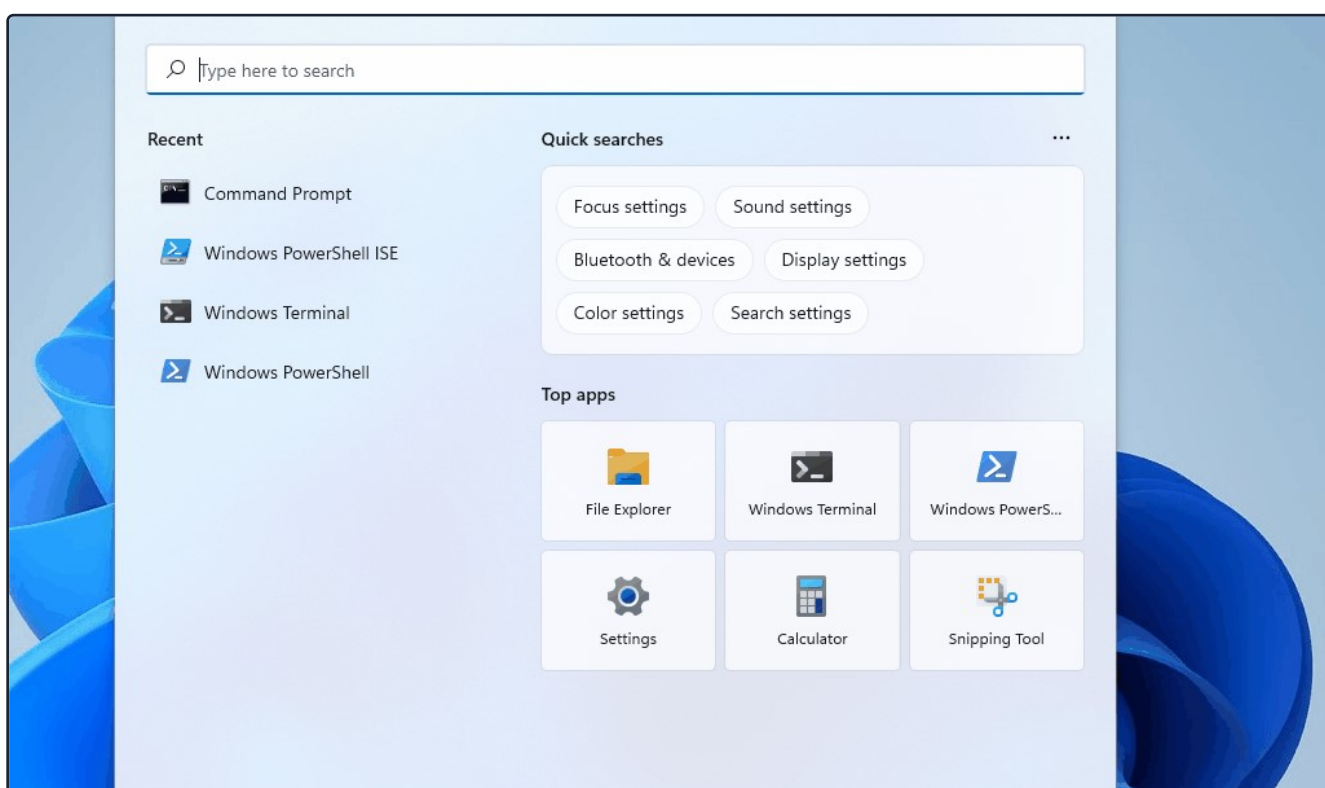
**Windows Terminal** is a newer terminal emulator application developed by Microsoft to give anyone using the Windows command line a way to access multiple different command line interfaces, systems, and sub-systems through a single app. This application will likely become the default terminal emulator on Windows operating systems.





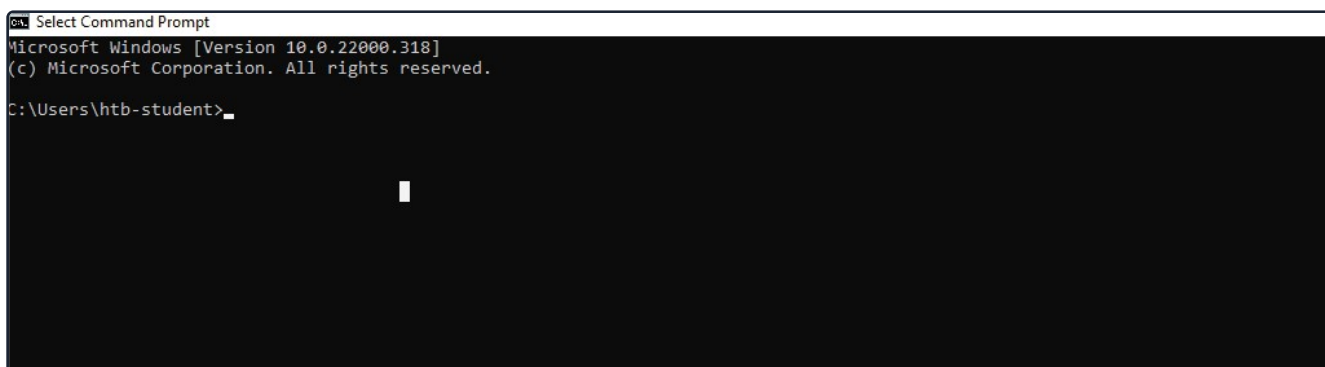
### 3. Using Windows PowerShell ISE

The [Windows PowerShell Integrated Scripting Environment \(ISE\)](#) is like an IDE for PowerShell. It can make it easier to develop, debug and test the PowerShell scripts we create. Using PowerShell ISE can be incredibly useful when learning PowerShell.



### 4. Using PowerShell in CMD

We can also launch PowerShell from within CMD. This action may seem trivial, but there will undoubtedly come a time when we can get a shell on a vulnerable Windows target's CLI via CMD and will benefit from attempting to use PowerShell to further our access on the host and across the network.





## Taking a Look at the Shell

One of the first things we may examine when accessing PowerShell on a local or remote system is the prompt.

### PowerShell Prompt

#### CMD Vs. PowerShell

```
PS C:\Users\htb-student> ipconfig

Ethernet adapter VMware Network Adapter VMnet8:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::adb8:3c9:a8af:114%25
    IPv4 Address. . . . . : 172.16.110.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :
```

The prompt is almost identical to what we see in CMD.

- **PS** is short for PowerShell, followed by the current working directory **C:\Users\htb-student>**.
- This is followed by the cmdlet or string we want to execute, **ipconfig**.
- Finally, below that, we see the output results of our command.

Also similar to CMD, PowerShell gives us many commands and cmdlets to utilize. Almost all commands that work in CMD will work in PowerShell. We will only cover some possible commands in this module. It is essential to understand that there is very little utility in memorizing commands. Focus more on understanding context, concepts, and what is possible. Memorization will naturally happen with time spent practicing and repetition.

### Get-Help

- Using the **Help** function. If we want to see the options and functionality available to us with a specific cmdlet, we can use the **Get-Help** cmdlet.

#### Using Get-Help

#### CMD Vs. PowerShell

```
PS C:\Users\htb-student> Get-Help Test-Wsman

NAME
    Test-WSMan

SYNTAX
    Test-WSMan [[-ComputerName] <string>] [-Authentication {None | Default | Digest | Negotiate | Basic | Kerb
```

```
ClientCertificate | Credssp}] [-Port <int>] [-UseSSL] [-ApplicationName <string>] [-Credential <pscredential>] [-CertificateThumbprint <string>] [<CommonParameters>]
```

**ALIASES**

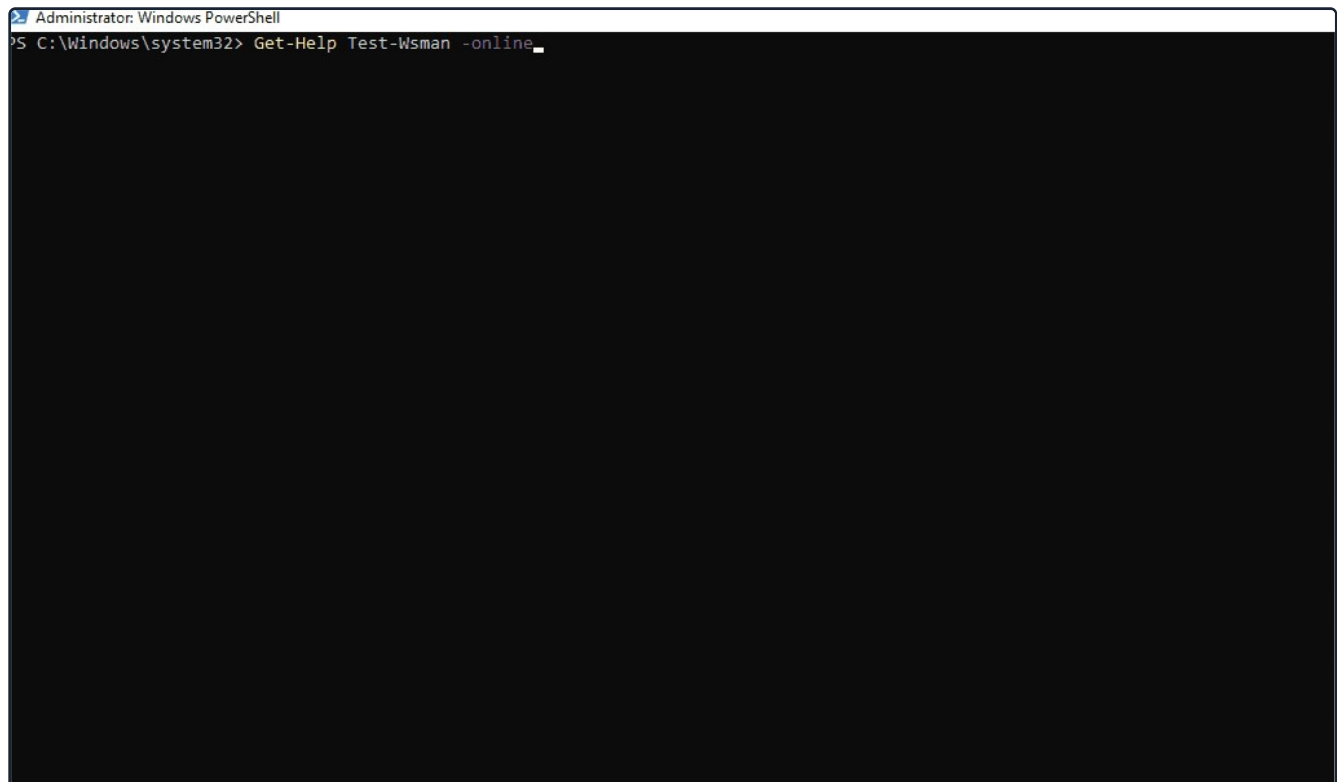
None

**REMARKS**

Get-Help cannot find the Help files for this cmdlet on this computer. It is displaying only partial help.

- To download and install Help files for the module that includes this cmdlet, use Update-Help.
- To view the Help topic for this cmdlet online, type: "Get-Help Test-WSMan -Online" or go to <https://go.microsoft.com/fwlink/?LinkId=141464>.

Get-Help can give helpful information about a cmdlet. Notice that the **Syntax** output shows us several available options and additional keywords that can be used with each option. **Aliases** are also mentioned, essentially shorter names for our commands. We will discuss Aliases in more depth later in this section. The **Remarks** output provides us with further information about the cmdlet and even additional options we can use to learn more about the cmdlet. One of these additional options is **-online**, which will open a Microsoft docs webpage for the corresponding cmdlet if the host has Internet access.



We can also use a helpful cmdlet called **Update-Help** to ensure we have the most up-to-date information for each cmdlet on the Windows system.

## Using Update-Help

**CMD Vs. PowerShell**

```
PS C:\Windows\system32> Update-Help
```

Notice how much more information was populated regarding **Test-WSman** after running **Update-Help**. Feel free to compare this output to the output shown earlier when we first covered Get-Help.

## Using Get-Help After Running Update-Help

| CMD Vs. PowerShell                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>PS C:\Windows\system32&gt; Get-Help Test-Wsman</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <p>NAME</p> <p>Test-WSMan</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <p>SYNOPSIS</p> <p>Tests whether the WinRM service is running on a local or remote computer.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <p>SYNTAX</p> <pre>Test-WSMan [[-ComputerName] &lt;System.String&gt;] [-ApplicationName &lt;System.String&gt;] [-Authentication {None   Default   Digest   Negotiate   Basic   Kerberos   ClientCertificate   Credssp}] [-CertificateThumbprint &lt;System.String&gt;] [-Credential &lt;System.Management.Automation.PSCredential&gt;] [-Port &lt;System.Int32&gt;] [-UseSSL] [&lt;CommonParameters&gt;]</pre>                                                                                                                                                                                                                                 |
| <p>DESCRIPTION</p> <p>The `Test-WSMan` cmdlet submits an identification request that determines whether the WinRM service is running on a local or remote computer. If the tested computer is running the service, the cmdlet displays the WS-Management identity schema, the protocol version, the product vendor, and the product version of the tested service.</p>                                                                                                                                                                                                                                                                         |
| <p>RELATED LINKS</p> <p>Online Version: <a href="https://docs.microsoft.com/powershell/module/microsoft.wsman.management/test-wsman?view=powershell-5.1&amp;WT.mc_id=ps-gethelp">https://docs.microsoft.com/powershell/module/microsoft.wsman.management/test-wsman?view=powershell-5.1&amp;WT.mc_id=ps-gethelp</a></p> <p>Connect-WSMan</p> <p>Disable-WSManCredSSP</p> <p>Disconnect-WSMan</p> <p>Enable-WSManCredSSP</p> <p>Get-WSManCredSSP</p> <p>Get-WSManInstance</p> <p>Invoke-WSManAction</p> <p>New-WSManInstance</p> <p>New-WSManSessionOption</p> <p>Remove-WSManInstance</p> <p>Set-WSManInstance</p> <p>Set-WSManQuickConfig</p> |
| <p>REMARKS</p> <p>To see the examples, type: "get-help Test-WSMan -examples".</p> <p>For more information, type: "get-help Test-WSMan -detailed".</p> <p>For technical information, type: "get-help Test-WSMan -full".</p> <p>For online help, type: "get-help Test-WSMan -online"</p>                                                                                                                                                                                                                                                                                                                                                         |

## Getting Around in PowerShell

Now that we have covered what PowerShell is and the basics of the built-in help features, let us get into basic navigation and usage of PowerShell.

### Where Are We?

We can only move around if we know where we are already, right? We can determine our current working directory (in relation to the host system) by utilizing the `Get-Location` cmdlet.

#### Get-Location

| CMD Vs. PowerShell                    |
|---------------------------------------|
| <pre>PS C:\htb&gt; Get-Location</pre> |

```
Path
----
C:\Users\DLarusso
```

We can see it printed the full path of the directory we are currently working from; in this case, that would be `C:\Users\DLarusso`. Now that we have our bearings let us look at what objects and files exist within this directory.

## List the Directory

The `Get-ChildItem` cmdlet can display the contents of our current directory or the one we specify.

### Get-ChildItem

#### CMD Vs. PowerShell

```
PS C:\htb> Get-ChildItem
```

```
Directory: C:\Users\DLarusso
```

| Mode   | LastWriteTime       | Length | Name       |
|--------|---------------------|--------|------------|
| ----   | -----               | -----  | ----       |
| d----- | 10/26/2021 10:26 PM |        | .ssh       |
| d----- | 1/28/2021 7:05 PM   |        | .vscode    |
| d-r--- | 1/27/2021 2:44 PM   |        | 3D Objects |
| d-r--- | 1/27/2021 2:44 PM   |        | Contacts   |
| d-r--- | 9/18/2022 12:35 PM  |        | Desktop    |
| d-r--- | 9/18/2022 1:01 PM   |        | Documents  |
| d-r--- | 9/26/2022 12:27 PM  |        | Downloads  |
| d-r--- | 1/27/2021 2:44 PM   |        | Favorites  |
| d-r--- | 1/27/2021 2:44 PM   |        | Music      |
| dar--l | 9/26/2022 12:03 PM  |        | OneDrive   |
| d-r--- | 5/22/2022 2:00 PM   |        | Pictures   |

We can see several other directories within our current working directory. Let's explore one.

## Move to a New Directory

Changing our location is simple; we can do so utilizing the `Set-Location` cmdlet.

### Set-Location

#### CMD Vs. PowerShell

```
PS C:\htb> Set-Location .\Documents\
```

```
PS C:\Users\tru7h\Documents> Get-Location
```

```
Path
----
C:\Users\DLarusso\Documents
```

We fed the parameters `.\Documents\` to the `Set-Location` cmdlet, telling PowerShell that we want to move into the Documents directory, which resides within our current working directory. We could have also given it the full file path like this:

Code: `powershell`

```
Set-Location C:\Users\DLarusso\Documents
```

## Display Contents of a File

Now, if we wish to see the contents of a file, we can use `Get-Content`. Looking in the Documents directory, we notice a file called `Readme.md`. Let us check it out.

### Get-Content

#### CMD Vs. PowerShell

```
PS C:\htb> Get-Content Readme.md

# ![logo][] PowerShell

Welcome to the PowerShell GitHub Community!
PowerShell Core is a cross-platform (Windows, Linux, and macOS) automation and configuration tool/framework that
for dealing with structured data (e.g., JSON, CSV, XML, etc.), REST APIs, and object models.
It includes a command-line shell, an associated scripting language and a framework for processing cmdlets.

<SNIP>
```

It looks like the Readme file was from the PowerShell GitHub page. Utilizing the `Get-Content` cmdlet is as simple as that. Navigating within the PowerShell CLI is pretty straightforward. Now that we have that skill down, let us look at a few helpful tips and tricks that can make utilizing the CLI even smoother.

## Tips & Tricks for PowerShell Usage

### Get-Command

`Get-Command` is a great way to find a pesky command that might be slipping from our memory right when we need to use it. With PowerShell using the `verb-noun` convention for cmdlets, we can search on either.

### Get-Command Usage

#### CMD Vs. PowerShell

```
PS C:\htb> Get-Command
```

| CommandType | Name                               | Version | Source       |
|-------------|------------------------------------|---------|--------------|
| -----       | ----                               | -----   | -----        |
| Alias       | Add-AppPackage                     | 2.0.1.0 | Appx         |
| Alias       | Add-AppPackageVolume               | 2.0.1.0 | Appx         |
| Alias       | Add-AppProvisionedPackage          | 3.0     | Dism         |
| Alias       | Add-ProvisionedAppPackage          | 3.0     | Dism         |
| Alias       | Add-ProvisionedAppxPackage         | 3.0     | Dism         |
| Alias       | Add-ProvisioningPackage            | 3.0     | Provisioning |
| Alias       | Add-TrustedProvisioningCertificate | 3.0     | Provisioning |
| Alias       | Apply-WindowsUnattend              | 3.0     | Dism         |
| Alias       | Disable-PhysicalDiskIndication     | 2.0.0.0 | Storage      |
| Alias       | Disable-StorageDiagnosticLog       | 2.0.0.0 | Storage      |
| Alias       | Dismount-AppPackageVolume          | 2.0.1.0 | Appx         |
| Alias       | Enable-PhysicalDiskIndication      | 2.0.0.0 | Storage      |
| Alias       | Enable-StorageDiagnosticLog        | 2.0.0.0 | Storage      |
| Alias       | Flush-Volume                       | 2.0.0.0 | Storage      |
| Alias       | Get-AppPackage                     | 2.0.1.0 | Appx         |

```
<SNIP>
```

The output above was snipped for the sake of saving screen space. Using `Get-Command` without additional modifiers will perform a complete output of each cmdlet currently loaded into the PowerShell session. We can trim this down more by filtering on the `verb` or



the **noun** portion of the cmdlet.

## Get-Command (verb)

| CMD Vs. PowerShell               |                              |         |                  |
|----------------------------------|------------------------------|---------|------------------|
| PS C:\htb> Get-Command -verb get |                              |         |                  |
| <SNIP>                           |                              |         |                  |
| Cmdlet                           | Get-Acl                      | 3.0.0.0 | Microsoft.Pow... |
| Cmdlet                           | Get-Alias                    | 3.1.0.0 | Microsoft.Pow... |
| Cmdlet                           | Get-AppLockerFileInformation | 2.0.0.0 | AppLocker        |
| Cmdlet                           | Get-AppLockerPolicy          | 2.0.0.0 | AppLocker        |
| Cmdlet                           | Get-AppvClientApplication    | 1.0.0.0 | AppvClient       |
| <SNIP>                           |                              |         |                  |

Using the **-verb** modifier and looking for any cmdlet, alias, or function with the term get in the name, we are provided with a detailed list of everything PowerShell is currently aware of. We can also perform the exact search using the filter **get\*** instead of the **-verb get**. The Get-Command cmdlet recognizes the **\*** as a wildcard and shows each variant of **get(anything)**. We can do something similar by searching on the noun as well.

## Get-Command (noun)

| CMD Vs. PowerShell                    |                                |         |                  |
|---------------------------------------|--------------------------------|---------|------------------|
| PS C:\htb> Get-Command -noun windows* |                                |         |                  |
| CommandType                           | Name                           | Version | Source           |
| -----                                 | ----                           | -----   | -----            |
| Alias                                 | Apply-WindowsUnattend          | 3.0     | Dism             |
| Function                              | Get-WindowsUpdateLog           | 1.0.0.0 | WindowsUpdate    |
| Cmdlet                                | Add-WindowsCapability          | 3.0     | Dism             |
| Cmdlet                                | Add-WindowsDriver              | 3.0     | Dism             |
| Cmdlet                                | Add-WindowsImage               | 3.0     | Dism             |
| Cmdlet                                | Add-WindowsPackage             | 3.0     | Dism             |
| Cmdlet                                | Clear-WindowsCorruptMountPoint | 3.0     | Dism             |
| Cmdlet                                | Disable-WindowsErrorReporting  | 1.0     | WindowsErrorR... |
| Cmdlet                                | Disable-WindowsOptionalFeature | 3.0     | Dism             |
| Cmdlet                                | Dismount-WindowsImage          | 3.0     | Dism             |
| Cmdlet                                | Enable-WindowsErrorReporting   | 1.0     | WindowsErrorR... |
| Cmdlet                                | Enable-WindowsOptionalFeature  | 3.0     | Dism             |

In the above output, we utilized the **-noun** modifier, took the filter a step further, and looked for any portion of the noun that contained **windows\***, so our results came up pretty specific. **Anything** that begins with windows in the noun portion and is followed by anything else would **match** this filter. These were just a few demonstrations of how powerful the **Get-Command** cmdlet can be. Paired with the **Get-Help** cmdlet, these can be powerful help functions provided to us directly by PowerShell. Our next tip dives into our PowerShell session History.

## History

PowerShell keeps a history of the commands run in two different ways. The first is the built-in session history which is implemented and deleted at the start and end of each console session. The other is through the **PSReadLine** module. The **PSReadLine** module tracks the history of any PowerShell commands used in all sessions across the host, among many other features. By default, PowerShell keeps the last 4096 commands entered, but this setting can be modified by changing the **\$MaximumHistoryCount** variable.

## Get-History

## CMD Vs. PowerShell

```
PS C:\htb> Get-History

Id CommandLine
--
1 Get-Command
2 clear
3 get-command -verb set
4 get-command set*
5 clear
6 get-command -verb get
7 get-command -noun windows
8 get-command -noun windows*
9 get-module
10 clear
11 get-history
12 clear
13 ipconfig /all
14 arp -a
15 get-help
16 get-help get-module
```

By default, `Get-History` will only show the commands that have been run during this active session. Notice how the commands are numbered; we can recall those commands by using the alias `r` followed by the number to run that command again. For example, if we wanted to rerun the `arp -a` command, we could issue `r 14`, and PowerShell will action it. Keep in mind that if we close the shell window, or in the instance of a remote shell through command and control, once we kill that session or process that we are running, our PowerShell history will disappear. With `PSReadLine`, however, that is not the case. `PSReadLine` stores everything in a file called `$(host.Name)_history.txt` located at `$env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine`.

## Viewing PSReadLine History

## CMD Vs. PowerShell

```
PS C:\htb> get-content C:\Users\DLarusso\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_h

get-module
Get-ChildItem Env: | ft Key,Value
Get-ExecutionPolicy
clear
ssh administrator@10.172.16.110.55
powershell -nop -c "iex(New-Object Net.WebClient).DownloadString('https://download.sysinternals.com/files/PSTool
Get-ExecutionPolicy

<SNIP>
```

If we ran the above command and were a frequent user of the CLI, we would have an extensive history file to sort through. The output above was snipped to save time and screen space. One great feature of `PSReadLine` from an admin perspective is that it will automatically attempt to filter any entries that include the strings:

- `password`
- `asplaintext`
- `token`
- `apikey`
- `secret`

This behavior is excellent for us as admins since it will help clear any entries from the `PSReadLine` history file that contain keys, credentials, or other sensitive information. The built-in session history does not do this.

## Clear Screen

This tip is one of convenience. If it bothers us to have a ton of output on our screen all the time, we can remove the text from our console window by using the command `Clear-Host`. It will only affect our current display and will not get rid of any variables or other objects we may have set or made during the session. We can also use `clear` or `cls` if we prefer using short commands or aliases.

## Hotkeys

Unless we are working in the CLI from a GUI environment, our mouse will **not** often work. For example, let's say we landed a `shell` on a host during a pentest. We will have access to CMD or PowerShell from this shell, but we will not be able to utilize the `GUI`. So we need to be comfortable using just a keyboard. **Hotkeys** can enable us to perform more complex actions that typically require a mouse with just our keys. Below is a quick list of some of the more useful hotkeys.

### Hotkeys

| HotKey                        | Description                                                                                                                                                     |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>CTRL+R</code>           | It makes for a searchable history. We can start typing after, and it will show us results that match previous commands.                                         |
| <code>CTRL+L</code>           | Quick screen clear.                                                                                                                                             |
| <code>CTRL+ALT+Shift+?</code> | This will print the entire list of keyboard shortcuts PowerShell will recognize.                                                                                |
| <code>Escape</code>           | When typing into the CLI, if you wish to clear the entire line, instead of holding backspace, you can just hit <code>escape</code> , which will erase the line. |
| <code>↑</code>                | Scroll up through our previous history.                                                                                                                         |
| <code>↓</code>                | Scroll down through our previous history.                                                                                                                       |
| <code>F7</code>               | Brings up a TUI with a scrollable interactive history from our session.                                                                                         |

This list is not all of the functionality we can use in PowerShell but those we find ourselves using the most.

## Tab Completion

One of PowerShell's best functionalities must be tab completion of commands. We can use `tab` and `SHIFT+tab` to move through options that can complete the command we are typing.

### Autocomplete Example

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

C:\Users\tru7h>
```

## Aliases

Our last tip to mention is [Aliases](#). A PowerShell alias is another name for a cmdlet, command, or executable file. We can see a list of default aliases using the [Get-Alias](#) cmdlet. Most built-in aliases are shortened versions of the cmdlet, making it easier to remember and quick to use.

### Using Get-Alias

#### CMD Vs. PowerShell

```
PS C:\Windows\system32> Get-Alias
```

| CommandType | Name                        | Version | Source |
|-------------|-----------------------------|---------|--------|
| Alias       | % -> ForEach-Object         |         |        |
| Alias       | ? -> Where-Object           |         |        |
| Alias       | ac -> Add-Content           |         |        |
| Alias       | asnp -> Add-PSSnapin        |         |        |
| Alias       | cat -> Get-Content          |         |        |
| Alias       | cd -> Set-Location          |         |        |
| Alias       | CFS -> ConvertFrom-String   | 3.1.0.0 | Mi...  |
| Alias       | chdir -> Set-Location       |         |        |
| Alias       | clc -> Clear-Content        |         |        |
| Alias       | clear -> Clear-Host         |         |        |
| Alias       | clhy -> Clear-History       |         |        |
| Alias       | cli -> Clear-Item           |         |        |
| Alias       | clp -> Clear-ItemProperty   |         |        |
| Alias       | cls -> Clear-Host           |         |        |
| Alias       | clv -> Clear-Variable       |         |        |
| Alias       | cnsn -> Connect-PSSession   |         |        |
| Alias       | compare -> Compare-Object   |         |        |
| Alias       | copy -> Copy-Item           |         |        |
| Alias       | cp -> Copy-Item             |         |        |
| Alias       | cpi -> Copy-Item            |         |        |
| Alias       | cpp -> Copy-ItemProperty    |         |        |
| Alias       | curl -> Invoke-WebRequest   |         |        |
| Alias       | cvpa -> Convert-Path        |         |        |
| Alias       | dbp -> Disable-PSBreakpoint |         |        |
| Alias       | del -> Remove-Item          |         |        |
| Alias       | diff -> Compare-Object      |         |        |
| Alias       | dir -> Get-ChildItem        |         |        |

<SNIP>

It is an excellent practice to make aliases shorter than the name of the actual cmdlet, command, or executable. Even the [Get-Alias](#) cmdlet has a default alias of [gaL](#), as seen in the clip below.

```
Administrator: Windows PowerShell
PS C:\Windows\system32>
```



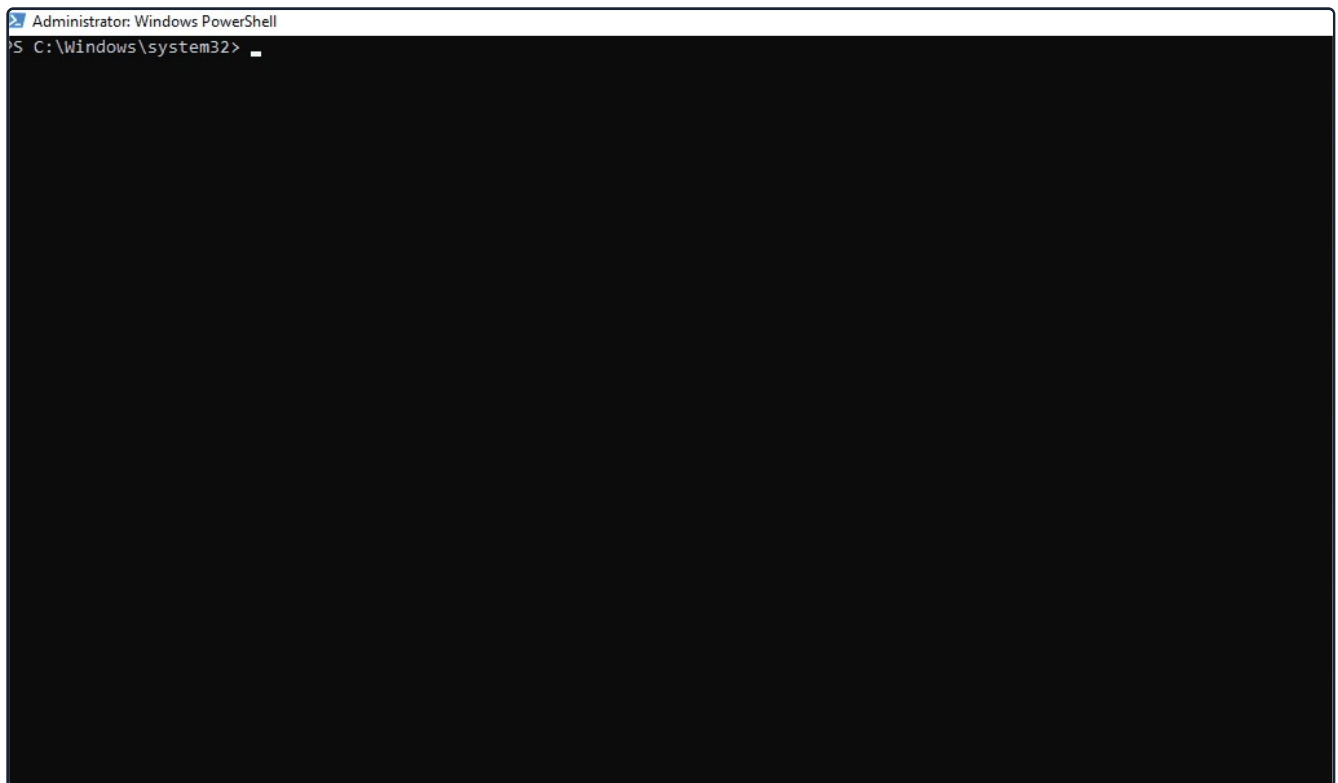
We can also set an alias for a specific cmdlet using [Set-Alias](#). Let us practice with this by making an alias for the `Get-Hello` cmdlet.

## Using Set-Alias

### CMD Vs. PowerShell

```
PS C:\Windows\system32> Set-Alias -Name gh -Value Get-Hello
```

When using `Set-Alias`, we need to specify the name of the alias (`-Name gh`) and the corresponding cmdlet (`-Value Get-Hello`).



Below we also include a list of several aliases we find to be most helpful. Some commands have more than one alias as well. Be sure to look at the complete list for other aliases you may find helpful.

## Helpful Aliases

| Alias            | Description                                                                                                                            |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>pwd</code> | <code>gl</code> can also be used. This alias can be used in place of <code>Get-Location</code> .                                       |
| <code>ls</code>  | <code>dir</code> and <code>gci</code> can also be used in place of <code>ls</code> . This is an alias for <code>Get-ChildItem</code> . |



|           |                                                                                                     |
|-----------|-----------------------------------------------------------------------------------------------------|
| cd        | sl and chdir can be used in place of cd. This is an alias for Set-Location.                         |
| cat       | type and gc can also be used. This is an alias for Get-Content.                                     |
| clear     | Can be used in place of Clear-Host.                                                                 |
| curl      | Curl is an alias for Invoke-WebRequest, which can be used to download files. wget can also be used. |
| fl and ft | These aliases can be used to format output into list and table outputs.                             |
| man       | Can be used in place of help.                                                                       |

For those familiar with **BASH**, you may have noticed that many of the aliases match up to commands widely used within Linux distributions. This knowledge can be helpful and help ease the learning curve.

This section has been a bit long, and for a good reason. We covered all the essentials to move us along our path to PowerShell mastery. From here, we will dive deep into PowerShell modules and cmdlets.

### VPN Servers

**Warning:** Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

### PROTOCOL

UDP 1337 TCP 443

DOWNLOAD VPN CONNECTION FILE



### Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

52ms

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

## Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)

[Cheat Sheet](#)[Download VPN  
Connection File](#)

+ 0

What command string can we use to view the help documentation for the command Get-Location? (full string)

Submit your answer here...

[Submit](#)[Hint](#)

+ 0

What command can we use to show us our current location on the host system?

Submit your answer here...

[Submit](#)[Hint](#)

+ 0

What hotkey can be used to clear our input line completely?

Submit your answer here...

[Submit](#)[Hint](#)[← Previous](#)[Next →](#)[Cheat Sheet](#)[Resources](#)[? Go to Questions](#)

## Table of Contents

Introduction



## CMD

Command Prompt Basics



Getting Help



|                                                                                                                      |                                                                                     |
|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  System Navigation                  |  |
|  Working with Directories and Files |  |
|  Gathering System Information       |  |
|  Finding Files and Directories      |  |
|  Environment Variables              |  |
|  Managing Services                  |  |
|  Working With Scheduled Tasks       |  |

## PowerShell

|                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------|
|  <a href="#">CMD Vs. PowerShell</a>    |
|  All About Cmdlets and Modules         |
|  User and Group Management             |
|  Working with Files and Directories    |
|  Finding & Filtering Content           |
|  Working with Services                 |
|  Working with the Registry             |
|  Working with the Windows Event Log    |
|  Networking Management from The CLI  |
|  Interacting with The Web            |
|  PowerShell Scripting and Automation |

## Finish Strong

|                                                                                                       |
|-------------------------------------------------------------------------------------------------------|
|  Skills Assessment |
|-------------------------------------------------------------------------------------------------------|

Beyond This Module

## My Workstation

OFFLINE

 Start Instance

 / 1 spawns left